

Set 10 - Constructors and Method Overloading

Q1.

Create a Java class called Product with three constructors: a default constructor, a parameterized constructor that takes name and price, and a copy constructor that copies another Product object. In your main method, create one Product using each constructor and print their details.

Answer:

```
public class Product {
    String name;
    double price;

    // Default constructor
    public Product() {
        this.name = "Unnamed Product";
        this.price = 0.0;
    }

    // Parameterized constructor
    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    // Copy constructor
    public Product(Product other) {
        this.name = other.name;
        this.price = other.price;
    }

    void display() {
        System.out.println("Name: " + name + ", Price: Rs. " + price);
    }

    public static void main(String[] args) {
        Product p1 = new Product();           // default
        Product p2 = new Product("Table Lamp", 799.0); // parameterized
        Product p3 = new Product(p2);         // copy

        p1.display();
        p2.display();
        p3.display();
    }
}

/* Sample Output:
Name: Unnamed Product, Price: Rs. 0.0
Name: Table Lamp, Price: Rs. 799.0
Name: Table Lamp, Price: Rs. 799.0
*/
```

Q2.

Build a class Movie with overloaded addReview methods: one that takes only a rating (int), and another that takes both a rating (int) and a review comment (String). Demonstrate adding reviews using both

methods for a Movie object.

Answer:

```
public class Movie {
    String title;

    public Movie(String title) {
        this.title = title;
    }

    // Overload 1: rating only
    void addReview(int rating) {
        System.out.println(title + " received a review - Rating: " + rating + "/5");
    }

    // Overload 2: rating + comment
    void addReview(int rating, String comment) {
        System.out.println(title + " received a review - Rating: " + rating + "/5, Comment: \"" + comment + "\"");
    }

    public static void main(String[] args) {
        Movie movie = new Movie("Interstellar");
        movie.addReview(5);
        movie.addReview(4, "Mind-bending and emotional!");
    }
}

/* Sample Output:
Interstellar received a review - Rating: 5/5
Interstellar received a review - Rating: 4/5, Comment: "Mind-bending and emotional!"
*/
```

Q3.

Simulate a Flipkart-style cart item insertion by creating a CartItem class with overloaded constructors: one for just productId, one for productId and quantity, and one that copies an existing CartItem. Show how each constructor could represent a different way of adding an item to the cart.

Answer:

```
public class CartItem {
    String productId;
    int quantity;

    // Constructor 1: just productId (default quantity = 1)
    public CartItem(String productId) {
        this.productId = productId;
        this.quantity = 1;
    }

    // Constructor 2: productId and quantity
    public CartItem(String productId, int quantity) {
        this.productId = productId;
        this.quantity = quantity;
    }

    // Constructor 3: copy constructor
    public CartItem(CartItem other) {
        this.productId = other.productId;
        this.quantity = other.quantity;
    }
}
```

```

void display() {
    System.out.println("Product ID: " + productId + ", Quantity: " + quantity);
}

public static void main(String[] args) {
    CartItem item1 = new CartItem("P1001");           // quick add, qty defaults to 1
    CartItem item2 = new CartItem("P1002", 3);        // add with specific quantity
    CartItem item3 = new CartItem(item2);            // duplicate an existing cart item

    item1.display();
    item2.display();
    item3.display();
}
}

/* Sample Output:
Product ID: P1001, Quantity: 1
Product ID: P1002, Quantity: 3
Product ID: P1002, Quantity: 3
*/

```

Q4.

Refactor the following code to use constructor chaining in the class Playlist, which has three constructors: default, one with name, and one with name and description. Ensure that each constructor calls another constructor using 'this()' where appropriate. Hint: Avoid repeating code for setting the name and description fields.

Answer:

```

public class Playlist {
    String name;
    String description;

    // Constructor with name and description - the "base" constructor
    public Playlist(String name, String description) {
        this.name = name;
        this.description = description;
        System.out.println("Playlist created: " + name);
    }

    // Constructor with name only - chains to the base constructor
    public Playlist(String name) {
        this(name, "No description provided");
    }

    // Default constructor - chains to the name-only constructor
    public Playlist() {
        this("Untitled Playlist");
    }

    void display() {
        System.out.println("Name: " + name + " | Description: " + description);
    }

    public static void main(String[] args) {
        Playlist p1 = new Playlist();
        Playlist p2 = new Playlist("Road Trip Mix");
        Playlist p3 = new Playlist("Study Beats", "Lo-fi tracks for focus");

        p1.display();
    }
}

```

```

        p2.display();
        p3.display();
    }
}

/* Sample Output:
Playlist created: Untitled Playlist
Playlist created: Road Trip Mix
Playlist created: Study Beats
Name: Untitled Playlist | Description: No description provided
Name: Road Trip Mix | Description: No description provided
Name: Study Beats | Description: Lo-fi tracks for focus
*/

```

Q5.

Imagine you are adding new ways to register users in a Zomato-style app. Use method overloading in a Registration class to create multiple registerUser methods: one with just email, one with email and password, and one with email, password, and phone number. Implement and call each method in your main method.

Answer:

```

public class Registration {

    // Overload 1: email only (e.g., magic-link style signup)
    void registerUser(String email) {
        System.out.println("User registered with email only: " + email);
    }

    // Overload 2: email and password
    void registerUser(String email, String password) {
        System.out.println("User registered with email: " + email + " and a password.");
    }

    // Overload 3: email, password, and phone number
    void registerUser(String email, String password, String phoneNumber) {
        System.out.println("User registered with email: " + email +
            ", password set, and phone number: " + phoneNumber);
    }

    public static void main(String[] args) {
        Registration reg = new Registration();
        reg.registerUser("user1@gmail.com");
        reg.registerUser("user2@gmail.com", "securePass1");
        reg.registerUser("user3@gmail.com", "securePass2", "9876543210");
    }
}

/* Sample Output:
User registered with email only: user1@gmail.com
User registered with email: user2@gmail.com and a password.
User registered with email: user3@gmail.com, password set, and phone number: 9876543210
*/

```